Quantium Retail Analytics – Task 1

Data Loading

We begin by importing the required libraries and loading the datasets. The transaction dataset contains product-level purchase information, while the customer dataset provides segmentation details.

```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving QVI_purchase_behaviour.csv to QVI_purchase_behaviour (1).csv
Saving QVI_transaction_data.xlsx to QVI_transaction_data (1).xlsx
```

The datasets have been successfully loaded. A preview confirms the presence of key variables such as transaction date, product details, and customer segmentation attributes.

Data Loading and Data Exploration

We begin by importing the required libraries and loading the datasets. The transaction dataset contains product-level purchase information, while the customer dataset provides segmentation details.

Next, we examine the structure of the datasets to understand data types and identify any issues.

```
import pandas as pd

transactions = pd.read_excel('QVI_transaction_data.xlsx')
customers = pd.read_csv('QVI_purchase_behaviour.csv')

print(transactions.head())
print(customers.head())
```

```
    DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0  43390          1            1000       1         5
1  43599          1            1307     348        66
2  43605          1            1343     383        61
3  43329          2            2373     974        69
4  43330          2            2426    1038       108

                                   PROD_NAME  PROD_QTY  TOT_SALES
0    Natural Chip        Compny SeaSalt175g         2        6.0
1                  CCs Nacho Cheese    175g         3        6.3
2    Smiths Crinkle Cut  Chips Chicken 170g         2        2.9
3    Smiths Chip Thinly  S/Cream&Onion 175g         5       15.0
```

```
4  Kettle Tortilla ChpsHny&Jlpno Chili 150g         3       13.8
   LYLTY_CARD_NBR               LIFESTAGE PREMIUM_CUSTOMER
0            1000   YOUNG SINGLES/COUPLES          Premium
1            1002   YOUNG SINGLES/COUPLES       Mainstream
2            1003          YOUNG FAMILIES           Budget
3            1004   OLDER SINGLES/COUPLES       Mainstream
4            1005  MIDAGE SINGLES/COUPLES       Mainstream
```

```python
# Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load files (since you uploaded via Method 1)
transactions = pd.read_excel('QVI_transaction_data.xlsx')
customers = pd.read_csv('QVI_purchase_behaviour.csv')

# Preview
print(transactions.head())
print(customers.head())

# Structure
print(transactions.info())
print(customers.info())
```

```
    DATE  STORE_NBR  LYLTY_CARD_NBR  TXN_ID  PROD_NBR  \
0  43390          1            1000       1         5
1  43599          1            1307     348        66
2  43605          1            1343     383        61
3  43329          2            2373     974        69
4  43330          2            2426    1038       108

                                  PROD_NAME  PROD_QTY  TOT_SALES
0    Natural Chip        Compny SeaSalt175g         2        6.0
1                  CCs Nacho Cheese    175g         3        6.3
2    Smiths Crinkle Cut  Chips Chicken 170g         2        2.9
3    Smiths Chip Thinly  S/Cream&Onion 175g         5       15.0
4  Kettle Tortilla ChpsHny&Jlpno Chili 150g         3       13.8
   LYLTY_CARD_NBR               LIFESTAGE PREMIUM_CUSTOMER
0            1000   YOUNG SINGLES/COUPLES          Premium
1            1002   YOUNG SINGLES/COUPLES       Mainstream
2            1003          YOUNG FAMILIES           Budget
3            1004   OLDER SINGLES/COUPLES       Mainstream
4            1005  MIDAGE SINGLES/COUPLES       Mainstream
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 264836 entries, 0 to 264835
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
```

```
 0   DATE            264836 non-null  int64
 1   STORE_NBR       264836 non-null  int64
 2   LYLTY_CARD_NBR  264836 non-null  int64
 3   TXN_ID          264836 non-null  int64
 4   PROD_NBR        264836 non-null  int64
 5   PROD_NAME       264836 non-null  object
 6   PROD_QTY        264836 non-null  int64
 7   TOT_SALES       264836 non-null  float64
dtypes: float64(1), int64(6), object(1)
memory usage: 16.2+ MB
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72637 entries, 0 to 72636
Data columns (total 3 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   LYLTY_CARD_NBR    72637 non-null  int64
 1   LIFESTAGE         72637 non-null  object
 2   PREMIUM_CUSTOMER  72637 non-null  object
dtypes: int64(1), object(2)
memory usage: 1.7+ MB
None
```

Both datasets appear complete with no missing values. The DATE column is stored as an integer and will be converted into a proper datetime format for analysis. Both datasets appear complete with no missing values. The DATE column is currently stored as an integer and will be converted.

Data Cleaning

The DATE column is converted into a datetime format to enable time-based analysis.

```
transactions['DATE'] = pd.to_datetime(transactions['DATE'],
origin='1899-12-30', unit='D')
```

The DATE column has been successfully converted.

We filter the dataset to include only chip products by removing non-relevant items such as salsa.

```
transactions =
transactions[~transactions['PROD_NAME'].str.lower().str.contains('sals
a')]
```

Non-chip products have been removed.

We review summary statistics to identify anomalies.

```
transactions.describe()
```

{"summary":"{\n \"name\": \"transactions\",\n \"rows\": 8,\n \"fields\": [\n {\n \"column\": \"DATE\",\n \"properties\": {\n \"dtype\": \"date\",\n \"min\": \"1970-01-01 00:00:00.000246742\",\n \"max\": \"2019-06-30 00:00:00\",\n \"num_unique_values\": 7,\n \"samples\": [\n \"246742\",\n \"2018-12-30 01:19:01.211467520\",\n \"2019-03-31 00:00:00\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"STORE_NBR\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 87191.66785025659,\n \"min\": 1.0,\n \"max\": 246742.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 135.05109790793622,\n 203.0,\n 246742.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"LYLTY_CARD_NBR\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 799130.4972569003,\n \"min\": 1000.0,\n \"max\": 2373711.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 135530.9849559459,\n 203084.0,\n 246742.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"TXN_ID\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 814125.7177510968,\n \"min\": 1.0,\n \"max\": 2415841.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 135131.0988481896,\n 202653.75,\n 246742.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"PROD_NBR\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 87217.73710731922,\n \"min\": 1.0,\n \"max\": 246742.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 56.351788507834094,\n 87.0,\n 246742.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"PROD_QTY\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 87225.91355659303,\n \"min\": 0.6598311218596009,\n \"max\": 246742.0,\n \"num_unique_values\": 6,\n \"samples\": [\n 246742.0,\n 1.9080618621880345,\n 0.6598311218596009\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"TOT_SALES\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 87202.2099803679,\n \"min\": 1.7,\n \"max\": 246742.0,\n \"num_unique_values\": 8,\n \"samples\": [\n 7.321322271846707,\n 8.8,\n 246742.0\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

An unusually high quantity (200 units) is identified as an outlier.

We remove the outlier customer to ensure realistic analysis.

```
outlier_customer = transactions[transactions['PROD_QTY'] == 200]
['LYLTY_CARD_NBR'].unique()

transactions =
transactions[~transactions['LYLTY_CARD_NBR'].isin(outlier_customer)]
```

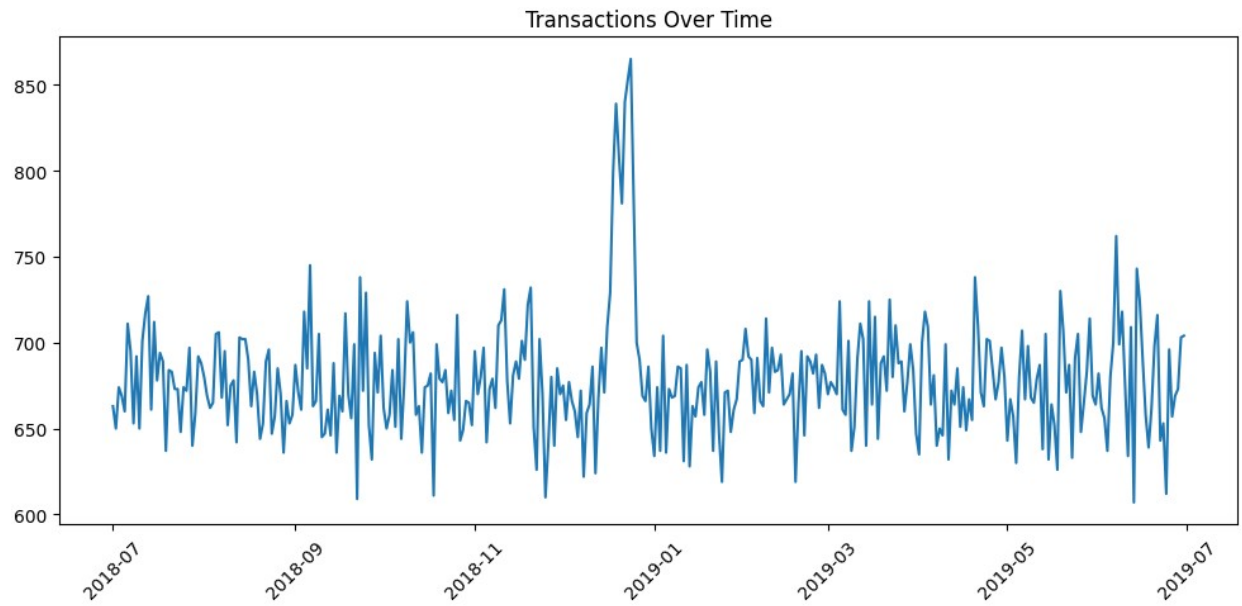
The dataset now reflects typical retail purchasing behaviour.

Time Series Analysis

We analyse transaction trends over time.

```
transactions_per_day =
transactions.groupby('DATE').size().reset_index(name='COUNT')

plt.figure(figsize=(12,5))
plt.plot(transactions_per_day['DATE'], transactions_per_day['COUNT'])
plt.title('Transactions Over Time')
plt.xticks(rotation=45)
plt.show()
```



Transactions peak in December, indicating seasonal demand, with a drop on Christmas Day due to store closures.

Feature Engineering

We extract pack size from product names.

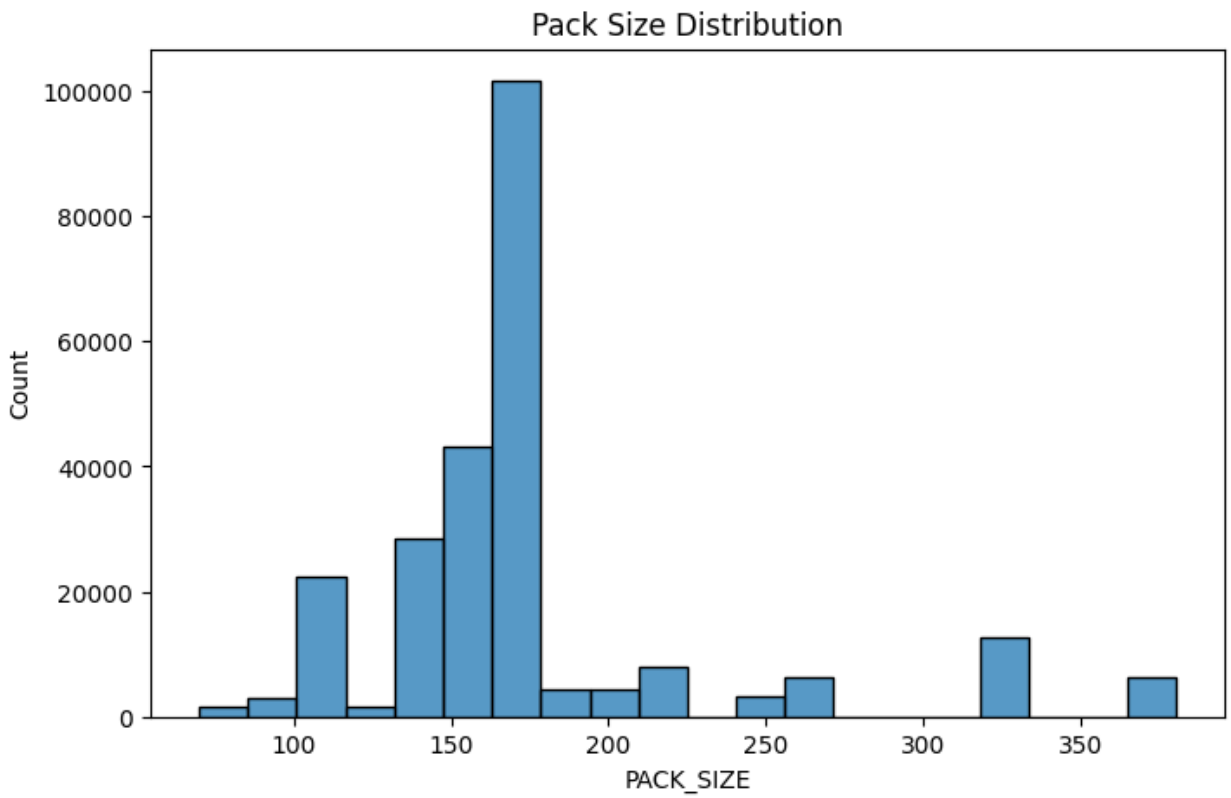
```
transactions['PACK_SIZE'] = transactions['PROD_NAME'].str.extract('(\
d+)').astype(float)
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_12499/3466836586.py:1: SyntaxWarning: invalid escape
sequence '\d'
  transactions['PACK_SIZE'] =
transactions['PROD_NAME'].str.extract('(\d+)').astype(float)
```

Pack size extraction is complete.

We examine pack size distribution.

```
plt.figure(figsize=(8,5))
sns.histplot(transactions['PACK_SIZE'], bins=20)
plt.title('Pack Size Distribution')
plt.show()
```



Most purchases fall between 150g–175g, with larger packs indicating family consumption.

We extract and clean brand names.

```
transactions['BRAND'] = transactions['PROD_NAME'].str.split().str[0]

transactions['BRAND'] = transactions['BRAND'].replace({
    'RED': 'RRD',
    'SNBTS': 'SUNBITES',
```

```
    'INFZNS': 'INFUZIONS'
})
```

Brand names are standardised for analysis.

Data Integration

We merge transaction and customer datasets.

```
data = pd.merge(transactions, customers, on='LYLTY_CARD_NBR',
how='left')

# Check nulls
print(data.isnull().sum())
```

```
DATE                0
STORE_NBR           0
LYLTY_CARD_NBR      0
TXN_ID              0
PROD_NBR            0
PROD_NAME           0
PROD_QTY            0
TOT_SALES           0
PACK_SIZE           0
BRAND               0
LIFESTAGE           0
PREMIUM_CUSTOMER    0
dtype: int64
```

The merged dataset contains no missing values.

Customer Segment Analysis

We create key metrics for analysis.

```
data['PRICE_PER_UNIT'] = data['TOT_SALES'] / data['PROD_QTY']
```

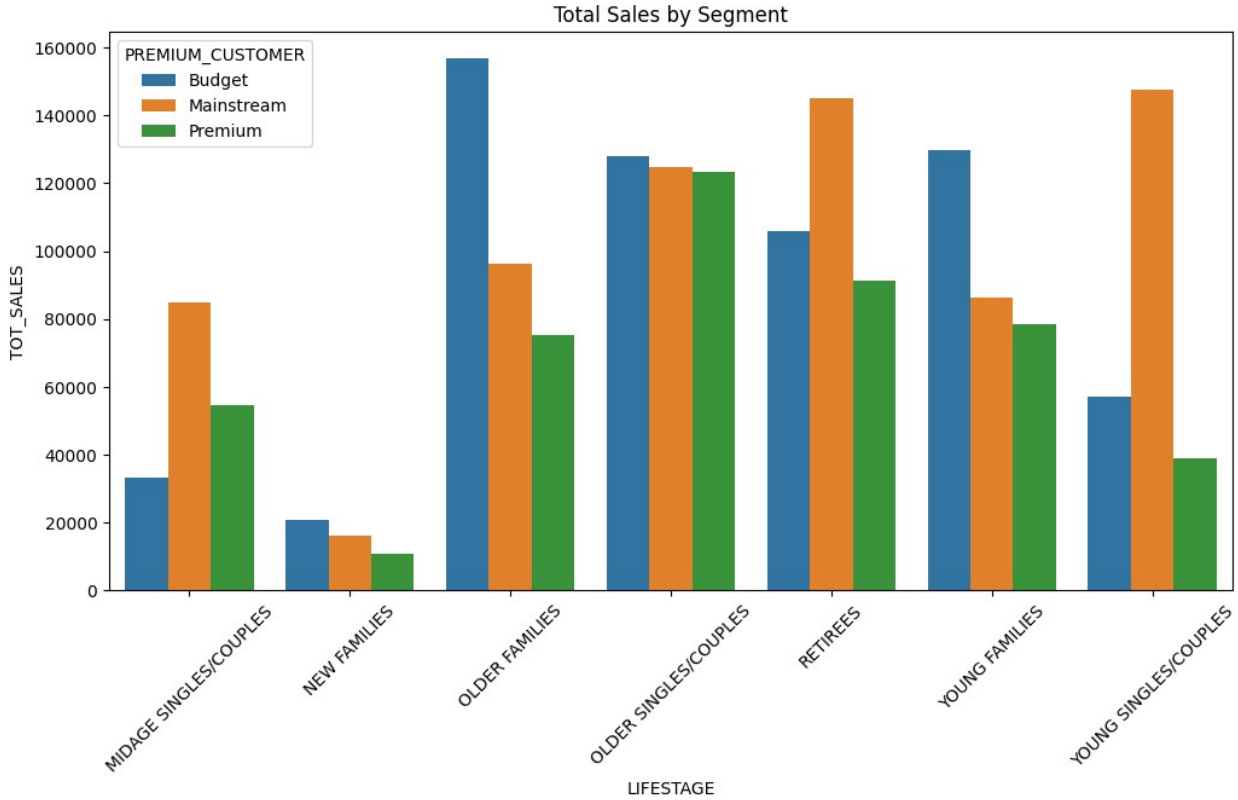
Metrics are ready for segmentation analysis.

Total Sales by Segment

```
sales_segment = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])
['TOT_SALES'].sum().reset_index()

plt.figure(figsize=(12,6))
sns.barplot(data=sales_segment, x='LIFESTAGE', y='TOT_SALES',
hue='PREMIUM_CUSTOMER')
plt.xticks(rotation=45)
```

```
plt.title('Total Sales by Segment')
plt.show()
```

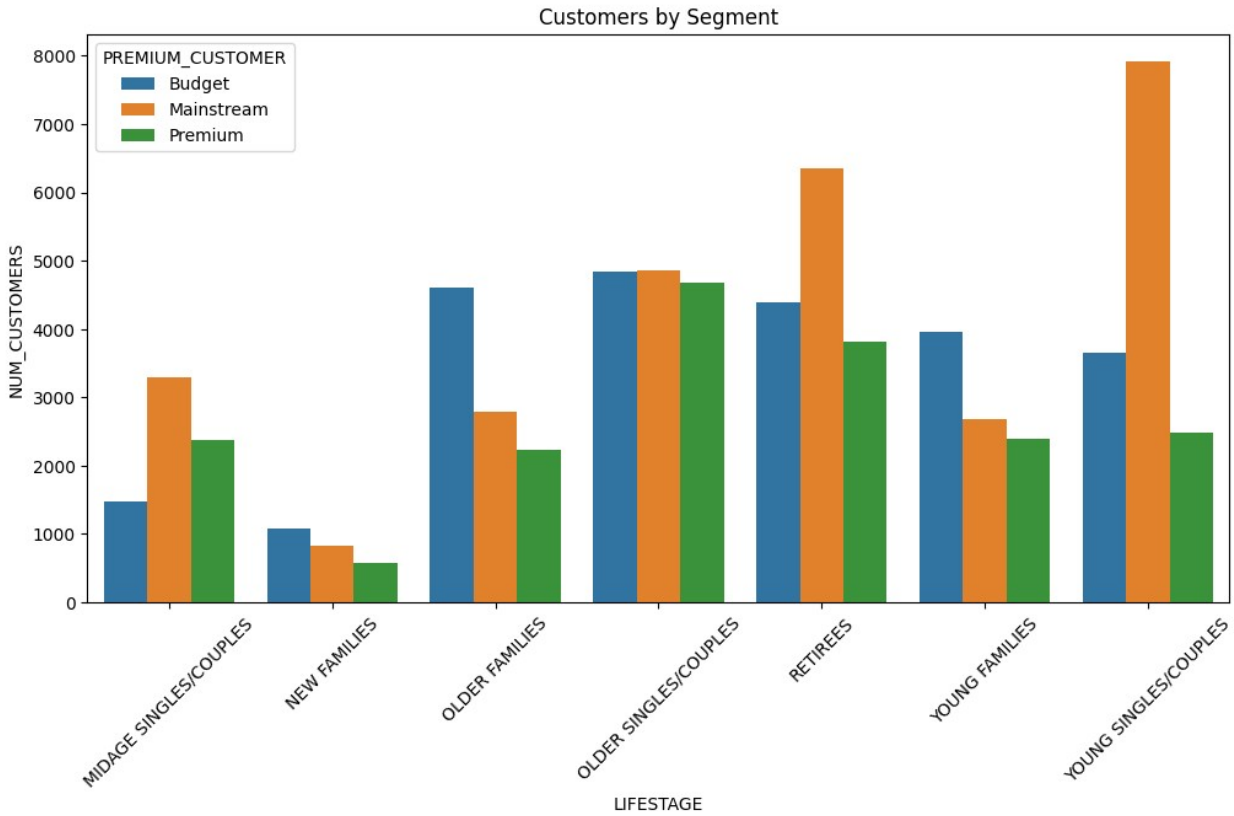


Mainstream young singles/couples and budget older families contribute the highest sales.

Customers by Segment

```
customers_segment = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])
['LYLTY_CARD_NBR'].nunique().reset_index()
customers_segment.rename(columns={'LYLTY_CARD_NBR': 'NUM_CUSTOMERS'},
inplace=True)

plt.figure(figsize=(12,6))
sns.barplot(data=customers_segment, x='LIFESTAGE', y='NUM_CUSTOMERS',
hue='PREMIUM_CUSTOMER')
plt.xticks(rotation=45)
plt.title('Customers by Segment')
plt.show()
```



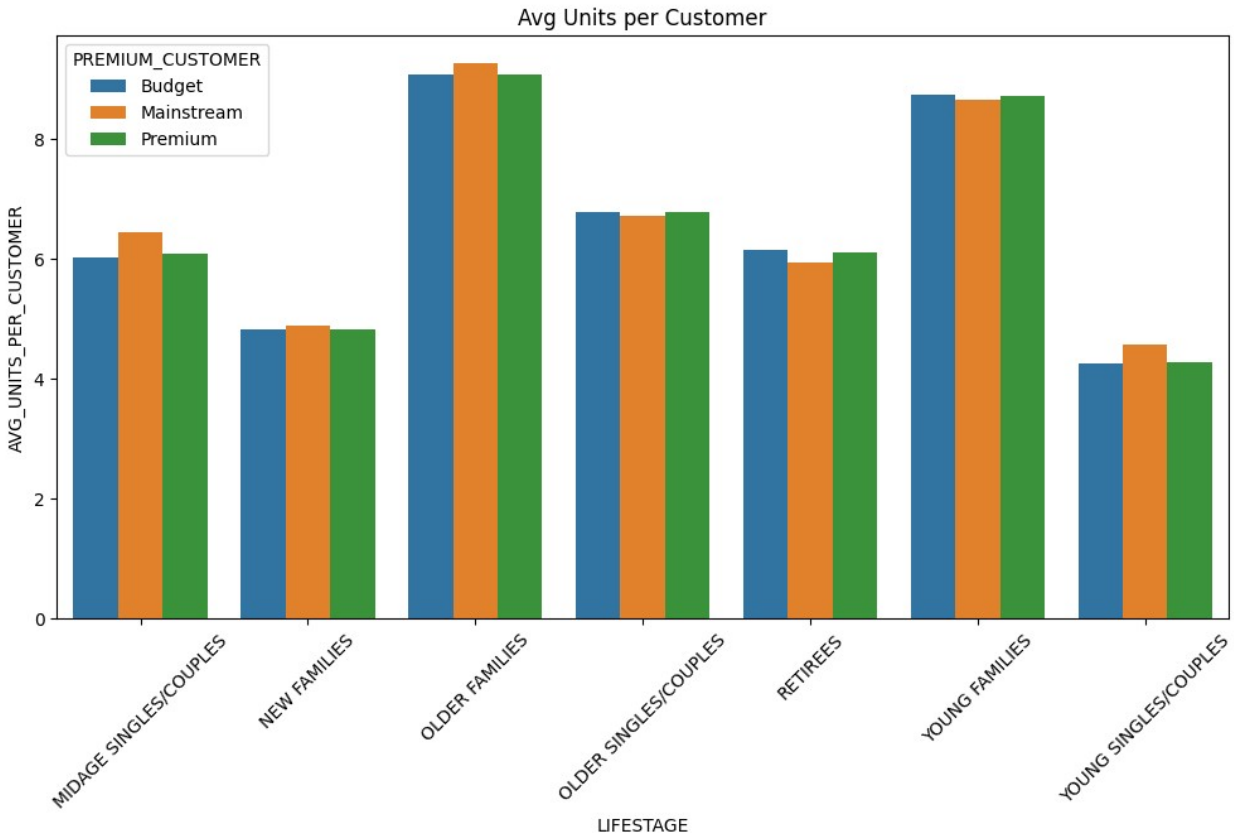
Mainstream segments dominate in customer count.

Units per Customer

```
units = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER']).agg({
    'PROD_QTY': 'sum',
    'LYLTY_CARD_NBR': 'nunique'
}).reset_index()

units['AVG_UNITS_PER_CUSTOMER'] = units['PROD_QTY'] /
units['LYLTY_CARD_NBR']

plt.figure(figsize=(12,6))
sns.barplot(data=units, x='LIFESTAGE', y='AVG_UNITS_PER_CUSTOMER',
hue='PREMIUM_CUSTOMER')
plt.xticks(rotation=45)
plt.title('Avg Units per Customer')
plt.show()
```

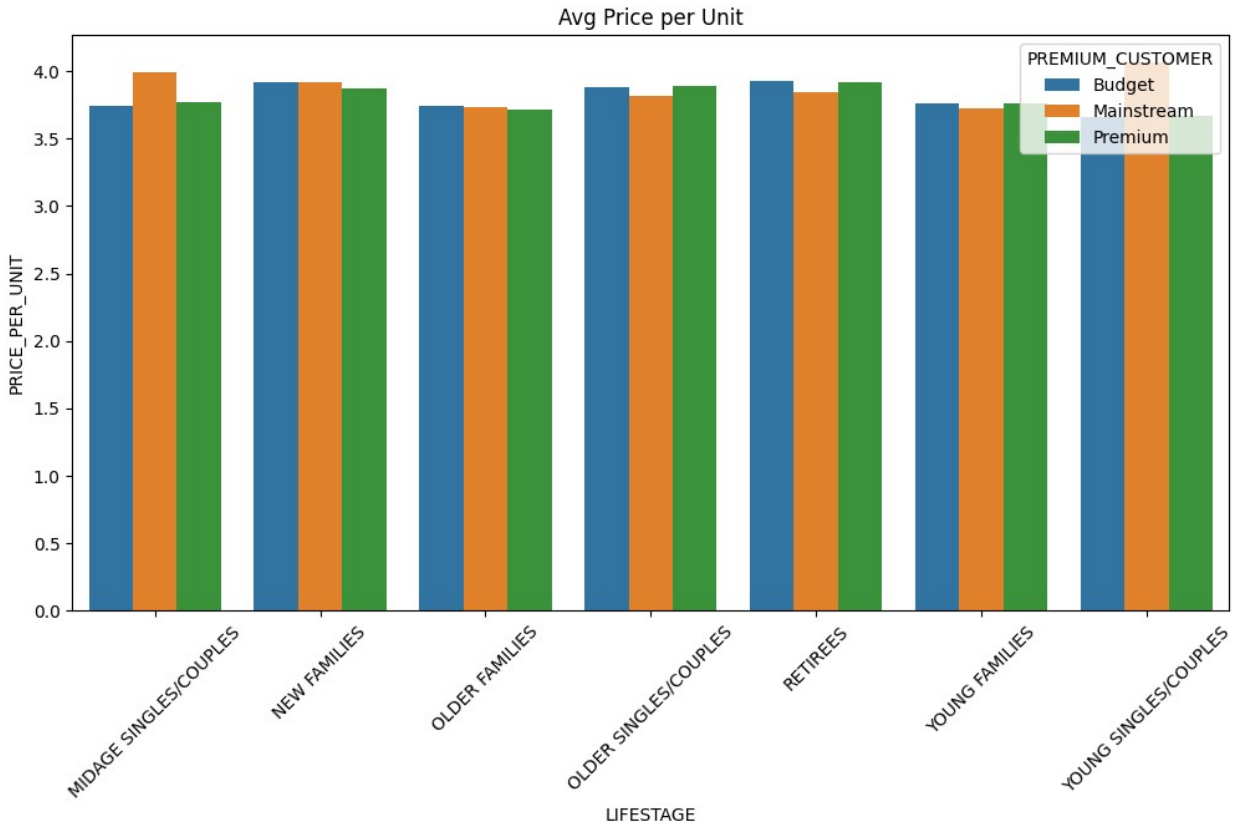


Families purchase the highest number of units, indicating bulk buying behaviour.

Price per Unit

```
price_segment = data.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])
['PRICE_PER_UNIT'].mean().reset_index()

plt.figure(figsize=(12,6))
sns.barplot(data=price_segment, x='LIFESTAGE', y='PRICE_PER_UNIT',
hue='PREMIUM_CUSTOMER')
plt.xticks(rotation=45)
plt.title('Avg Price per Unit')
plt.show()
```



Price differences are minimal across segments, suggesting behaviour drives sales.

Deep Dive: Target Segment

We focus on Mainstream Young Singles/Couples.

```
target = data[
    (data['LIFESTAGE'] == 'YOUNG SINGLES/COUPLES') &
    (data['PREMIUM_CUSTOMER'] == 'Mainstream')
]

others = data.drop(target.index)
```

This segment is selected due to its strong contribution to sales.

Brand Affinity

```
target_brand = target['BRAND'].value_counts(normalize=True)
other_brand = others['BRAND'].value_counts(normalize=True)

brand_affinity = pd.concat([target_brand, other_brand], axis=1)
brand_affinity.columns = ['Target', 'Other']
brand_affinity['Affinity'] = brand_affinity['Target'] /
brand_affinity['Other']
```

```
brand_affinity.sort_values(by='Affinity', ascending=False).head()
```

{"summary":"{\n \"name\": \"brand_affinity\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"BRAND\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"Twisties\",\n \"Tostitos\",\n \"Dorito\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Target\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.03434130028739027,\n \"min\": 0.015503479328694229,\n \"max\": 0.1062218583708555,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.046049938600081865,\n 0.04553827261563651,\n 0.015503479328694229\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Other\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.028526966324958325,\n \"min\": 0.01267627951196324,\n \"max\": 0.08787566682512016,\n \"num_unique_values\": 5,\n \"samples\": [\n 0.03765031074490748,\n 0.037769150865332134,\n 0.01267627951196324\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"Affinity\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 0.01219334904000764,\n \"min\": 1.2057001963852876,\n \"max\": 1.23575132811724,\n \"num_unique_values\": 5,\n \"samples\": [\n 1.223095844071101,\n 1.2057001963852876,\n 1.2230307255423658\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

This segment shows stronger preference for popular and flavour-driven brands.

Pack Size Preference

```
target_pack = target['PACK_SIZE'].value_counts(normalize=True)
other_pack = others['PACK_SIZE'].value_counts(normalize=True)

pack_affinity = pd.concat([target_pack, other_pack], axis=1)
pack_affinity.columns = ['Target', 'Other']
pack_affinity['Affinity'] = pack_affinity['Target'] /
pack_affinity['Other']

pack_affinity.sort_values(by='Affinity', ascending=False).head()
```

{"summary":"{\n \"name\": \"pack_affinity\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"PACK_SIZE\",\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 97.03195349986518,\n \"min\": 134.0,\n \"max\": 380.0,\n

\"num_unique_values\": 5,\n   \"samples\": [\n   380.0,\n
210.0,\n   330.0\n   ],\n   \"semantic_type\": \"\",\
n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"Target\",\n   \"properties\": {\n   \"dtype\":
\"number\",\n   \"std\": 0.0380259164720648 9,\n   \"min\":
0.029471960704052395,\n   \"max\": 0.11845067539909947,\n
\"num_unique_values\": 5,\n   \"samples\": [\n
0.03203029062627916,\n   0.029471960704052395,\n
0.061144085141219814\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"Other\",\n   \"properties\": {\n   \"dtype\": \"number\",\n
\"std\": 0.03264436440993252,\n   \"min\": 0.024934417859469357,\
n   \"max\": 0.10029666015246748,\n   \"num_unique_values\":
5,\n   \"samples\": [\n   0.025484603602176095,\n
0.02507086392366063,\n   0.04993485800806352\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"Affinity\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\":
0.04354950844575482,\n   \"min\": 1.1755462753015955,\n
\"max\": 1.2722691668361683,\n   \"num_unique_values\": 5,\n
\"samples\": [\n   1.2568486889685873,\n
1.1755462753015955,\n   1.2244770002419196\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   }\n]\n}","type":"dataframe"}

Preference for smaller to mid-sized packs suggests convenience-driven purchasing.

Output

```
data.to_csv('cleaned_data.csv', index=False)
```

The dataset is saved for further analysis.